

Lab 7 - Configuration Management Plan

This plan outlines how our team the evolving software system for our fitness tracking app. Given our tight 3-week timeline and PHP/MySQL stack, consistent CM is vital to prevent code conflicts.

Part 1: Version Management

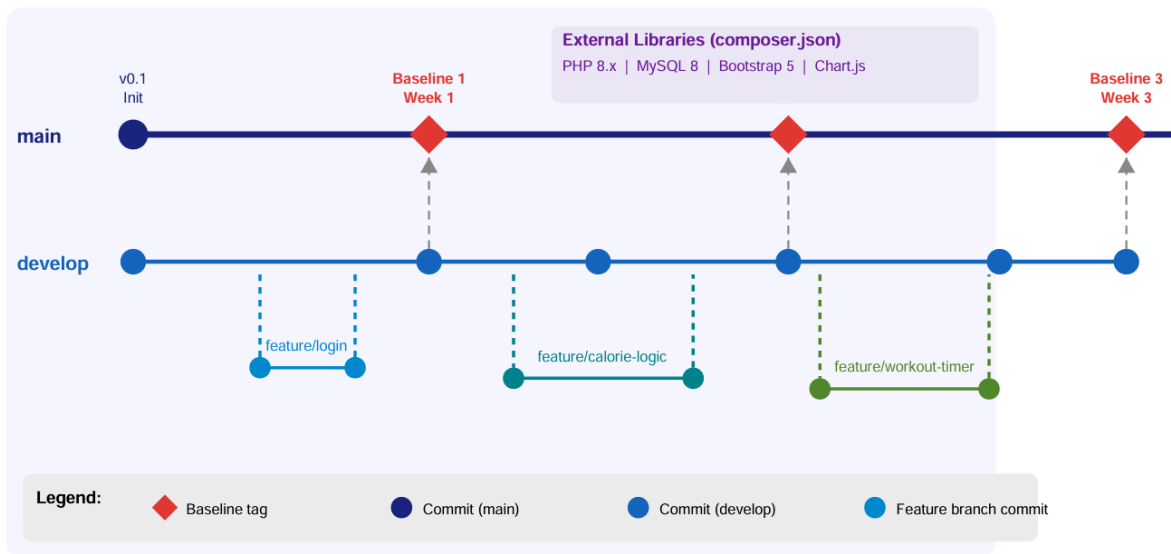
1. Deployment Strategies

- **GitHub Setup:** We utilize a **Feature Branch Workflow**.
 - **Branches:** The main branch is protected and always holds stable, deployable code. The develop branch serves as the integration point. For every user story (e.g., "Calorie Tracking"), a developer creates a new branch (e.g., feature/calorie-logic).
 - **Check-in/Check-out:** We use Git for local development. Developers "pull" the latest develop code before starting work to minimize merge conflicts.
 - **Pull Requests (PRs):** No code enters develop or main without a PR. At least one other developer must review the code for logic errors and "code smells" before approval.
- **Monitoring Resources:**
 - **GitHub:** Our primary source for version history and code reviews.
 - **Discord:** Integrated with GitHub webhooks to notify the team instantly when a PR is opened or code is merged.
 - **Google Drive:** Used for non-code assets, such as the Configuration Management Plan and Risk Register.

2. Baselines

Our project establishes baselines at the end of each weekly sprint.

- **Baseline 1 (End of Week 1):** Documentation and Database Schema finalized.
- **Baseline 2 (End of Week 2):** Core MVP (Registration/Login) functional and merged to main.
- **Baseline 3 (End of Week 3):** Final release with all features and bug fixes.



Part 2: System Building

To ensure our PHP/MySQL environment remains stable, we follow a tiered testing process:

1. **Development Environment:** Developers test locally using XAMPP or MAMP to ensure their specific feature (e.g., the Workout Timer JS) works in isolation.
2. **Integration Testing:** Once a Feature Branch is merged into develop, we perform "smoke tests" to ensure the new code hasn't broken existing features (like the Login logic).
3. **Build Validation:** Before the final baseline is tagged, the Scrum Master (Matthew) performs a full system build on a clean environment to ensure no hard-coded local paths were used and the database seeds correctly.

Part 3: Change Management

1. Change Request Process

If a client (or Product Owner) requests a change, we follow this workflow:

- **Documentation:** A **Change Request Form** must be completed, detailing:
 - Requester Name & Date
 - Description of Change
 - Priority (Low, Medium, High)
 - Impact Analysis (How will this affect our 3-week deadline?)

- **Approval:** The Product Owner (Shondel) reviews the request. If it threatens the 3-week MVP, it is moved to the "Future Backlog." If approved, the Scrum Master updates the Sprint Backlog.

2. Revision History

This table tracks changes to our Configuration Management, Risk Management, and Acceptance Test documents.

Version	Date	Description	Author
1.0	Feb 15, 2026	Initial Risk Management Plan created	Shondel
1.1	Feb 22, 2026	Updated Risk R5 (Resource availability)	Matthew
2.0	Mar 03, 2026	Full Configuration Management Plan drafted	Shondel

Part 4: Release Management

Our final project configuration release will include a **Release Package** consisting of:

- **Executables:** The full source code folder (HTML/CSS/JS/PHP).
- **Database:** A .sql export file to recreate the MySQL schema and "Food/Exercise" seeds.
- **Documentation:** * User Manual for app navigation.
 - Final Acceptance Test Report showing all 10 stories passed.
 - Installation Guide (PHP version requirements and DB connection setup).
- **Code Standards:** A README confirming all code has been refactored for readability and follows the PSR (PHP Standard Recommendation).